Лабораторная работа №5

Аппарат сетей Петри

Российский университет дружбы народов, Москва, Россия

::: :::::::::::::
::: :::::::::::::

Цель работы

- Построим сеть Петри для пяти философов, моделируя захват и освобождение вилок. Обнаружим состояние взаимной блокировки (deadlock), когда каждый философ взял одну вилку и ждёт вторую. Провести имитационное моделирование (стохастическое и детерминированное) и выявить наличие deadlock. Модифицировать сеть, чтобы предотвратить deadlock. Проанализировать результаты и оформить отчёт с графиками и анимацией.

Задание

1. Создать рабочий каталог для кода.
2. Установить необходимые пакеты.
3. Выполнить предложенный код.
4. Преобразовать код в литературный стиль.
5. Сгенерировать из литературного кода:

- чистый код;
- jupyter notebook;
- документацию в формате Quarto.

6. Выполнить код из jupyter notebook.
7. Интегрировать документацию в формате Quarto в отчёт.
8. Добавить в код в литературном стиле вычисление для набора параметров.
9. Сгенерировать из литературного кода с параметрами:

- чистый код;

Выполнение лабораторной работы

Обедающие философы

- Мы создадим файл src/DiningPhilosophers.jl и запустим его.

Ouvrir

DiningPhilosophers.jl

~/work/study/2025-2026/simulation-modeling/study_2025-2026_simulation-modeling/labs/lab05/project/src

DiningPhilosophers.jl | dining_philosophers.jl | dining_philosophers_animation.jl

```julia
module DiningPhilosophers

using OrdinaryDiffEq
using Plots
using DataFrames
using Random, LinearAlgebra

export build_classical_network, build_arbiter_network
export simulate_ode, simulate_stochastic
export detect_deadlock, plot_marking_evolution

# Определение простой структуры PetriNet
struct PetriNet
    n_places::Int
    n_transitions::Int
    incidence::Matrix{Int}
    place_names::Vector{Symbol}
    transition_names::Vector{Symbol}
end

function PetriNet(
    n_places,
    n_transitions;
    place_names = Symbol[],
    transition_names = Symbol[],
```

- Вывод в консоль.

```
julia> touch("src/DiningPhilosophers.jl")
"src/DiningPhilosophers.jl"

julia> include("src/DiningPhilosophers.jl")
Main.DiningPhilosophers
```

Рис. 2: Рис

Базовый эксперимент

- Скрипт выполняет основное моделирование и сравнение двух вариантов сети Петри:

1. классическая модель (без арбитра), в которой возможна взаимная блокировка (deadlock);
2. модифицированная модель с арбитром, которая должна предотвращать deadlock.



Рис. 3: Рис

Базовый эксперимент

- Создание файла scripts/dining_philosophers.jl и запустим его.

```
julia> include("scripts/dining_philosophers.jl")
  Activating project at `~/work/study/2025-2026/simulation-modeling/study_2025-2026_simulation-modeling/labs/lab05/project`
WARNING: replacing module DiningPhilosophers.
=== Классическая сеть (без арбитра) ===
Deadlock обнаружен: true

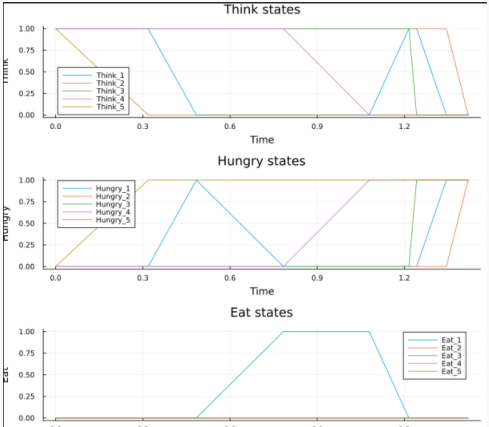
=== Сеть с арбитром ===
Could not create decoration from factory! Running with no decorations.
Deadlock обнаружен: false
"/home/serazanacua/work/study/2025-2026/simulation-modeling/study_2025-2026_simulation-modeling/labs/lab05/project/plots/arbiter_simulation.png"
```

Рис. 4: Рис

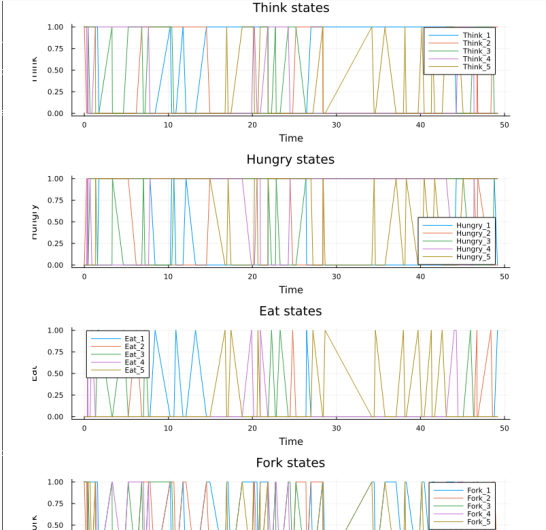
Базовый эксперимент

- Графики эволюции маркировки позволяют визуально увидеть, как меняются состояния.

1. В классической сети рано или поздно все Eat_i становятся нулевыми, а Hungry_i – единичными (или наоборот).



2. В сети с арбитром наблюдается постоянное чередование приёма пищи.



Анимация процесса

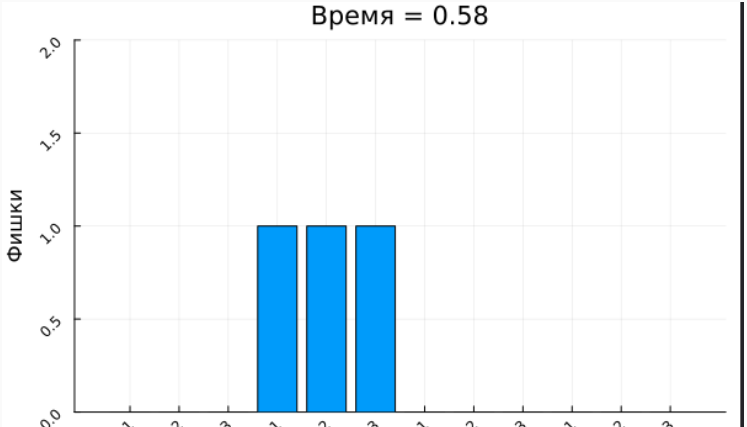
- Для наглядной демонстрации динамики работы сети Петри во времени.
- Анимация позволяет увидеть, как меняется маркировка (фишки) в каждой позиции, и особенно наглядно показывает возникновение deadlock в классической модели.



Рис. 7: Рис

Анимация процесса

- Анимация делает абстрактные состояния сети Петри живыми.
- Видно, как фишки перемещаются между позициями: философы переходят из Think в Hungry, затем в Eat и обратно.



Итоговый отчёт

- Для сравнительного анализа двух моделей (с арбитром и без) по одному ключевому показателю — числу философов, находящихся в состоянии «Ест» (Eat_i).

- Скрипт генерирует сводный график.



Рис. 9: Рис

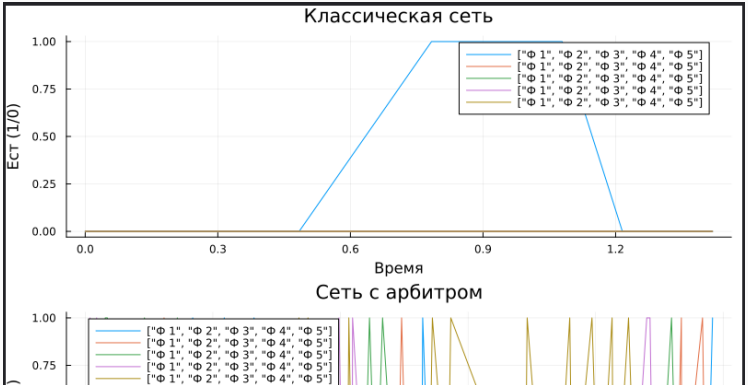
Итоговый отчёт

1. Классическая сеть: на графике видно, что через некоторое время все линии Eat_i падают до нуля и остаются на нуле

- Это и есть deadlock.
- Философы больше не могут есть.

2. Сеть с арбитром: линии Eat_i колеблются, всегда есть хотя бы один философ, который ест (или они поочерёдно получают доступ).

- Отсутствие длительных нулевых периодов подтверждает, что система жива и не блокируется.



Выводы

- Я выполнила лабораторную работу.